

Scribed by:

1. Aman Divya (2022MT11293)
2. Aditya Raj (2022MT61980)
3. Ayush Raj (2022MT11944)
4. Dev Singh (2022MT11143)
5. Kartik Kumar Singh (2022MT61569)

1 Overview

In the last lecture we covered Rice's theorem — its statement, proof, and applications. In this lecture we turn to the **complexity theory of Turing machines**: how efficiently different TM models solve problems, whether some models are fundamentally better than others, and in what sense they are (or are not) equivalent.

2 Introduction: What is Complexity Theory?

Core Idea: We have different models of Turing Machines (e.g., single-tape, multi-tape). In terms of computability, they are all equivalent (they can solve the same set of problems). The difference is their *efficiency* (complexity), which can be very different.

Running Time $f(n)$: For a deterministic decider Turing Machine (DTM) M , the *running time* or *time complexity* of M is a function $f : \mathbb{N} \rightarrow \mathbb{N}$ defined as follows: for each input length n , $f(n)$ is the maximum number of steps that M takes on any input string of length n before halting. This represents the **worst-case running time** of M .

Note: We restrict our discussion of time complexity to *decider* deterministic Turing machines only, since for non-deciders (machines that may not halt on some inputs), the running time is not well-defined.

3 Model Equivalence and Simulation (DTMs)

The specific model of DTM we choose can affect the running time, but usually only by a polynomial amount.

Example: Consider the language $L = \{0^n 1^n \mid n \geq 0\}$.

- A **multi-tape DTM** can solve this in $O(n)$ (linear) time.
- A **single-tape DTM** requires $O(n^2)$ (quadratic) time.

Claim 1 (Simulation of k -tape by a single-tape DTM). *Let M be a k -tape deterministic Turing Machine that performs at most $t(n)$ steps on any input of length n . Then there exists an equivalent single-tape deterministic Turing Machine S that simulates M in time $O(t^2(n))$. In particular, a two-tape machine running for n steps can be simulated by a standard single-tape machine in $O(n^2)$ steps.*

Proof. The proof idea is to represent all k tapes of M on one tape of S and simulate one step of M by several passes of S over its tape.

- 1. Tape Encoding:** The single tape of S stores the contents of all k tapes of M concatenated together, with a special delimiter symbol (say ‘#’) between them. Each cell under a head of M is represented by a *marked* version of its symbol to indicate the head position.
- 2. Tape Size:** After $t(n)$ steps, M can visit at most $t(n)$ new cells on each tape. Hence, the total encoded length of S ’s tape is $O(k \cdot t(n)) = O(t(n))$ since k is constant.
- 3. Simulating One Step of M :** To simulate a single step of M , the single-tape machine S performs:
 - 1. Scan Phase:** Traverse the tape to locate the k marked cells and record the symbols seen by M ’s heads in its finite control.
 - 2. Update Phase:** Traverse again to:
 - write the new symbols as per M ’s transition function, and
 - move each head mark one cell left or right, possibly shifting symbols if the head crosses a delimiter.

Each full traversal takes $O(t(n))$ time since the tape length is $O(t(n))$.

- 4. Time for Simulation:** Simulating one step of M requires $O(t(n))$ time on S . Since M executes $t(n)$ steps in total,

$$T_S(n) = t(n) \cdot O(t(n)) = O(t^2(n)).$$

- 5. Special Case:** For a two-tape machine that halts in n steps, the simulation time on a single-tape machine is $O(n^2)$.

This completes the proof. □

4 Non-Deterministic Turing Machines (NDTM)

This is a different theoretical model where the machine can have multiple possible moves from a single configuration.

Running Time of an NDTM: A non-deterministic machine N is a decider with running time $f(n)$ if $f(n)$ is the **maximum** number of steps taken on **any branch** of its computation for an input of length n . This is the *depth* of the computation tree.

Claim 2 (Simulating NDTM by a DTM). *Let N be a non-deterministic Turing machine that runs at most $t(n)$ steps on every input of length n . There exists an equivalent deterministic single-tape Turing machine D that decides the same language in time $2^{O(t(n))}$.*

Proof.

Preliminaries and Equivalences

- A non-deterministic Turing Machine (NDTM) can be simulated by a deterministic **multi-tape TM** without changing the language it accepts.
- From the previous theorem, a multi-tape deterministic TM can be simulated by a **single-tape** deterministic TM with only a quadratic slowdown ($O(t^2(n))$).

Computation Tree View

- The computation of N on input x can be visualized as a rooted tree of configurations.
- Each node represents a configuration, and each edge represents one nondeterministic move.
- Let b be the maximum branching factor (number of possible next moves).
- The depth of the tree is at most $t(n)$ (the number of computation steps).
- Hence, the total number of possible computation paths is at most $b^{t(n)}$.

Deterministic Simulation Idea (Breadth-First Search)

- The DTM D performs a breadth-first traversal of the computation tree of N .
- It systematically explores all configurations at depth 1, then depth 2, and so on up to $t(n)$.
- For each configuration:
 - Simulate one step of N deterministically.
 - If an accepting configuration is found, **accept**.

Time Analysis

- Each configuration has size $O(t(n))$ (since N can use at most $t(n)$ tape cells in $t(n)$ steps).
- Simulating one configuration and generating its successors takes $O(t(n))$ time.
- Total number of configurations: $O(b^{t(n)})$.

- Therefore, the total simulation time on a deterministic *multi-tape* TM (e.g., 3-tape) is

$$T_{3\text{-tape}}(n) = O(t(n) \cdot b^{t(n)}).$$

(Here $t(n)$ is the time per configuration, and $b^{t(n)}$ is the total number of configurations explored.)

- Converting this to a *single-tape* deterministic TM introduces a quadratic slowdown. From the standard simulation theorem:

$$T_{\text{single-tape}}(n) = O((T_{3\text{-tape}}(n))^2) = O((t(n) \cdot b^{t(n)})^2) = O(t(n)^2 \cdot b^{2t(n)}).$$

- Simplifying the exponential term: since b is a fixed constant,

$$b^{2t(n)} = (b^2)^{t(n)} = 2^{t(n) \log_2(b^2)} = 2^{O(t(n))}.$$

The polynomial factor $t(n)^2$ is negligible compared to the exponential term, so

$$T_{\text{single-tape}}(n) = t(n)^2 \cdot (b^2)^{t(n)} = 2^{O(t(n))}.$$

Conclusion

$$T_{3\text{-tape}}(n) = O(t(n) b^{t(n)}) \implies T_{\text{single-tape}}(n) = O((t(n) b^{t(n)})^2) = 2^{O(t(n))}.$$

□

Intuition (One-line Summary) Each nondeterministic path has length at most $t(n)$, the number of paths is bounded by $b^{t(n)}$, and checking each path costs polynomial time, giving a total of $t(n) b^{t(n)} = 2^{O(t(n))}$.

Homework Question

Question. Can we design a deterministic Turing machine (DTM) corresponding to a non-deterministic Turing machine (NDTM) whose running time is polynomial?

Answer. In general, this is an open question known as the *P vs NP* problem. A straightforward deterministic simulation of a polynomial-time NDTM requires exponential time in the worst case, since it must explore all possible computation branches. Converting multi-tape to single-tape DTMs adds only polynomial overhead, so it does not change this exponential behavior. Hence, whether every polynomial-time NDTM can be simulated by a polynomial-time DTM remains unresolved.

5 Decision vs. Optimization Problems

Complexity theory primarily focuses on **decision problems**, which have a simple "yes/no" answer.

Example: Maximum Independent Set (MIS)

- **Optimization Problem:** Given a graph G , what is the size of the *largest* independent set? (Answer is a number.)
- **Decision Problem:** Given a graph G and an integer k , does G have an independent set of size *at least* k ? (Answer is yes/no.)

Why focus on decision problems?

- They are simpler to define and reason about mathematically.
- The complexity of the decision version usually captures the essential difficulty of the optimization version.
- If the decision problem can be solved efficiently, then the optimization problem can also be solved efficiently.

From Optimization to Decision Let us define the language

$$L = \{\langle G, k \rangle : G \text{ has an independent set of size at least } k\}.$$

If there exists a Turing Machine M that decides L in polynomial time, then we can use M as a subroutine to solve the optimization version by performing a **binary search** on k . Since k ranges between 1 and $|V(G)|$, this binary search takes $O(\log |V(G)|)$ calls to M .

Conclusion Thus, optimization problems can be efficiently reduced to their equivalent decision problems. Therefore, in complexity theory, we primarily study decision problems without losing generality.

6 Language Families and Complexity Classes

We now define several fundamental classes of languages based on the running time of Turing Machines.

Definition 3 (**TIME**($t(n)$)). The class **TIME**($t(n)$) consists of all languages that can be **decided** by a **single-tape** deterministic Turing Machine (DTM) in **time** $O(t(n))$.

$$\text{TIME}(t(n)) = \{ L \mid \exists \text{ a single-tape DTM } M \text{ that decides } L \text{ in } O(t(n)) \text{ time} \}.$$

Definition 4 (The Class **P**). The class **P** (**Polynomial Time**) is defined as the **union of all deterministic time classes bounded by a polynomial**.

$$\mathbf{P} = \bigcup_{k \geq 1} \text{TIME}(n^k)$$

That is, **P** contains all languages that can be decided by some deterministic Turing Machine in polynomial time.

Definition 5 ($\text{NTIME}(t(n))$). The class $\text{NTIME}(t(n))$ consists of all languages that can be decided by a **nondeterministic Turing Machine (NDTM)** in time $O(t(n))$.

$$\text{NTIME}(t(n)) = \{ L \mid \exists \text{ an NDTM } M \text{ that decides } L \text{ in } O(t(n)) \text{ time} \}.$$

Definition 6 (The Class **NP**). The class **NP** (Nondeterministic Polynomial Time) is the union of all nondeterministic time classes bounded by a polynomial:

$$\mathbf{NP} = \bigcup_{k \geq 1} \text{NTIME}(n^k)$$

That is, **NP** contains all languages decidable by some nondeterministic Turing Machine in polynomial time.

Verifier-Based Definition of NP

Definition 7 (Verifier). A **verifier** for a language L is a deterministic Turing Machine V such that:

$$x \in L \iff \exists y \text{ (certificate) with } V(x, y) = 1.$$

The machine V runs in polynomial time in the length of x , and y is called a certificate or proof of membership.

Definition 8 (NP via Verifiers). A language L belongs to **NP** if there exists a polynomial-time verifier V and a polynomial $p(n)$ such that:

$$x \in L \iff \exists y, |y| \leq p(|x|), \text{ and } V(x, y) = 1.$$

Equivalently,

$$\mathbf{NP} = \{ L \mid \exists \text{ a poly-time verifier for } L \}.$$

Intuition: For inputs $x \in L$, a *certificate* y can be thought of as a "hint" that helps V efficiently check that x is indeed in L . For example, for the *Hamiltonian Cycle* problem, y could be the actual cycle.

Claim 9. A language has a polynomial-time verifier **if and only if** it has a polynomial-time nondeterministic Turing Machine.

Proof Sketch. ($\Rightarrow$) If a language L has a polynomial-time verifier V , then an NDTM M can guess the certificate y nondeterministically and then run $V(x, y)$ deterministically in polynomial time. Hence $L \in \mathbf{NP}$.

($\Leftarrow$) Conversely, if L has a polynomial-time NDTM M , then a deterministic verifier V can take (x, y) as input, where y encodes the sequence of nondeterministic choices of M . V simulates M deterministically following those choices, accepting if M would accept. This runs in polynomial time.

Thus, the two definitions of **NP** are equivalent. □

References

- [1] P. Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett Learning, 2016.